

AI Hedge Fund Research & Trading Agent Complete Project Blueprint

This document describes a full frontier-style AI systems project suitable for a course focused on modern AI systems, autonomous agents, simulation environments, evaluation loops, and AI deployment workflows. The proposed system is a multi-agent autonomous financial research and trading platform. The system simulates historical markets while ensuring strict point-in-time information integrity. Multiple AI agents analyze market conditions, generate research reports, reason about risk, and produce trading recommendations. A trading agent then allocates capital based on the outputs of the research system. The project emphasizes: - agentic workflows, - explainable reasoning, - historical simulation, - temporal data integrity, - evaluation, - iterative learning, - autonomous decision systems, - and realistic market environments. The system is NOT intended to become a profitable hedge fund. Instead, the goal is to demonstrate strong systems thinking, autonomous reasoning, simulation architecture, evaluation rigor, and frontier-style AI workflows.

1. High-Level Project Goal

Build an autonomous multi-agent financial research and trading system capable of operating within a historically accurate market simulation.

The system should:

1. Ingest only information available before a specific point in time.
2. Analyze historical market conditions.
3. Produce natural language financial reasoning.
4. Produce structured quantitative trading signals.
5. Execute trades in a simulated environment.
6. Evaluate performance over time.
7. Adapt or reflect on prior mistakes.

The project should resemble a lightweight autonomous hedge fund research workflow rather than a traditional “stock prediction model.”

2. Core System Architecture

The project should be divided into five major subsystems:

A. Historical Data Layer

- Market prices
- News
- Earnings
- Financial statements
- Macroeconomic indicators

B. Simulation Environment

- Point-in-time replay engine
- Portfolio accounting
- Order execution
- Transaction costs
- Position tracking

C. Research Agent Layer

- News analyst agent
- Technical analysis agent
- Macro analyst agent
- Risk agent
- Aggregator agent

D. Trading Layer

- Portfolio construction
- Position sizing
- Trade execution
- Risk management

E. Evaluation Layer

- Benchmarking
 - Performance metrics
 - Failure analysis
 - Strategy comparison
-

3. Recommended Technology Stack

Programming Language:

- Python

Core Libraries:

- pandas
- numpy
- matplotlib
- scikit-learn
- langchain or llamaindex
- transformers
- openai SDK
- alpaca-py
- backtesting.py or vectorbt

Optional Libraries:

- chromadb or faiss
- streamlit
- fastapi
- redis

Frontend:

- Streamlit dashboard OR React frontend

Infrastructure:

- Local development initially
 - Optional deployment on DigitalOcean
-

4. Data Sources

Market Data:

- Alpaca
- Polygon.io
- Yahoo Finance
- Alpha Vantage

News Data:

- Finnhub
- NewsAPI
- GDELT

Fundamental Data:

- SEC EDGAR
- Financial Modeling Prep

Macroeconomic Data:

- FRED API

Important:

ALL data must be constrained by historical timestamp.

The system must never access future information during simulation.

5. Historical Market Simulator Design

This is one of the most important parts of the project.

The simulator acts as the “environment” in which the AI agents operate.

The simulator should:

- progress chronologically,
- reveal data only when historically available,
- execute trades,
- update portfolio state,
- track PnL,
- and compute evaluation metrics.

Simulation Loop:

FOR each trading day:

1. Load historical market state
2. Load news released before current timestamp
3. Load macro indicators available at current time
4. Send context to research agents
5. Collect recommendations
6. Send recommendations to trading agent
7. Execute simulated trades
8. Advance to next timestep
9. Record metrics

This historical replay environment is critical for preventing temporal leakage.

6. Preventing Future Information Leakage

This is arguably the single most important technical challenge.

The system must avoid:

- future news,
- future earnings,
- revised economic data,
- future company descriptions,
- survivorship bias,
- future analyst ratings,
- and future market constituents.

Examples of leakage:

BAD:

Using current S&P500; constituents for a 2018 simulation.

BAD:

Using a 2026 summary of a company while simulating 2020.

BAD:

Allowing the agent to see post-earnings price movement before making a decision.

Solutions:

- timestamp all data,
 - enforce strict cutoff dates,
 - maintain chronological replay,
 - use point-in-time datasets,
 - avoid manually curated future knowledge.
-

7. Multi-Agent Research System

The research system should consist of multiple specialized agents.

Example Agents:

A. News Analysis Agent

Purpose:

- analyze headlines,
- summarize events,
- determine sentiment,
- identify catalysts.

B. Technical Analysis Agent

Purpose:

- analyze price action,
- detect momentum,
- identify volatility patterns.

C. Macro Agent

Purpose:

- analyze interest rates,

- inflation,
- economic indicators,
- recession probability.

D. Risk Agent

Purpose:

- identify concentration risk,
- monitor volatility,
- evaluate drawdown exposure.

E. Aggregator Agent

Purpose:

- combine outputs,
- resolve disagreements,
- generate final thesis.

This architecture makes the project feel much more frontier-oriented.

8. Research Agent Outputs

The research agent should produce BOTH:

1. Natural Language Reports

Example:

"NVDA appears bullish due to strong AI demand growth, positive earnings momentum, and increasing institutional interest."

2. Structured Quantitative Signals

Example JSON:

```
{  
  "ticker": "NVDA",  
  "bullish_score": 0.82,  
  "confidence": 0.76,  
  "risk_score": 0.31,  
  "suggested_action": "BUY"  
}
```

This dual-output architecture is highly recommended.

9. Trading Agent Design

The trading agent should consume research outputs and convert them into executable portfolio decisions.

Responsibilities:

- position sizing,
- capital allocation,
- stop loss logic,
- rebalancing,
- exposure management,

- execution timing.

Suggested Rules:

- max 10% capital per position,
- max sector exposure limits,
- volatility-adjusted sizing,
- confidence-weighted allocations.

The trading agent should NOT simply follow every recommendation blindly.

10. Portfolio Simulation

The system should track:

- cash balance,
- positions,
- unrealized gains,
- realized gains,
- transaction history,
- leverage,
- risk exposure.

Transaction Costs:

You MUST model:

- commissions,
- slippage,
- spread costs.

Otherwise results become unrealistic.

11. Evaluation Metrics

The project should include rigorous evaluation.

Minimum Metrics:

- Total return
- Sharpe ratio
- Max drawdown
- Volatility
- Win rate
- Alpha vs benchmark

Benchmarks:

- SPY buy-and-hold
- QQQ buy-and-hold
- Random strategy
- Momentum strategy

Additional Analysis:

- performance during crises,
- regime sensitivity,

- failure cases,
 - calibration quality.
-

12. Reflection and Learning

This is where the project becomes especially frontier-style.

The system should maintain memory of failures.

Example:

“Sentiment signals overreacted during high-volatility periods.”

Possible Improvements:

- dynamic weighting,
- reflection loops,
- memory store,
- adaptive confidence scaling,
- self-critique.

This creates:

- feedback loops,
 - agentic traces,
 - iterative adaptation.
-

13. Recommended Development Timeline

WEEK 1:

- Define architecture
- Set up repositories
- Gather market data

WEEK 2:

- Build historical replay engine
- Implement portfolio simulator

WEEK 3:

- Build news ingestion pipeline
- Implement timestamp filtering

WEEK 4:

- Build research agents
- Implement LLM prompting

WEEK 5:

- Build trading agent
- Add portfolio allocation logic

WEEK 6:

- Add evaluation metrics
- Add benchmark comparisons

WEEK 7:

- Add memory/reflection system
- Improve explainability

WEEK 8:

- Build frontend/dashboard

WEEK 9:

- Run experiments
- Tune prompts and parameters

WEEK 10:

- Final demo
 - README
 - Video presentation
-

14. Suggested MVP Scope

To avoid scope explosion, the MVP should initially support:

- daily timestep simulation,
- 10–20 stocks,
- one research agent,
- one trading agent,
- news + price data only.

Once stable:

- add more agents,
 - add fundamentals,
 - add reflection,
 - add live paper trading.
-

15. Final Demo Ideas

Excellent demo possibilities include:

1. Replay the March 2020 COVID crash
Show how the agents reacted in real time.
2. Replay major earnings events
Example:
 - NVIDIA earnings
 - Tesla earnings
3. Compare against SPY
Show performance differences.
4. Show agent reasoning live
Display:
 - headlines,

- agent analysis,
- trade decisions,
- portfolio evolution.

This makes the project visually compelling and understandable.

16. Final Deliverables

Final deliverables should include:

1. Public GitHub repository
 2. Reproducible instructions
 3. Evaluation notebook
 4. Architecture diagram
 5. Demo video
 6. README with:
 - AI usage disclosure
 - limitations
 - data sources
 - architecture explanation
 7. Historical simulation results
 8. Benchmark comparisons
-

17. Potential Stretch Goals

Optional advanced features:

- Live Alpaca paper trading
 - Multi-agent debate system
 - Reinforcement learning policy optimization
 - Vector database memory
 - Long-term strategy adaptation
 - Multi-asset trading
 - Crypto support
 - Options trading
 - Real-time deployment
 - Voice-based analyst assistant
-

18. Final Recommendation

The strongest version of this project is not:
“AI predicts stocks.”

Instead, frame it as:

“A multi-agent autonomous financial research and trading system operating inside a historically accurate constrained-information simulation environment.”

That framing:

- sounds significantly more ambitious,
 - aligns with frontier systems themes,
 - emphasizes agents and environments,
 - demonstrates evaluation rigor,
 - and creates a strong final presentation.
-